자료구조 HW4 보고서

공과대학 컴퓨터공학부
2020-17316
최준우

1. 정렬 알고리즘 작동 방식

1.1. Bubble Sort

```java
    private static int[] DoBubbleSort(int[] value)
    {
        for (int i=value.length-1; i>0; i--) {
            for (int j=0; j<i; j++) {
                if (value[j] > value[j+1]) {
                    swap(value, j, j+1);
                }
            }
        }

        return (value);
    }
```

Bubble Sort 에서는 배열의 i 번째 자리(오름차순 정렬에서는 맨 뒤부터 하나씩 앞으로)에, 배열의 i 번째 이전까지 값들의 최댓값을 넣는다. 위 코드에서 j 는 0 부터 i 전까지의 값을 가지며 증가한다. 배열의 j 번째 값과 j+1 번째 값을 비교하여, 앞의 값이 뒤의 값보다 작으면 교환한다. i=value.length-1 부터 i=1 까지의 모든 자리에 최댓값이 오면 정렬이 끝난다. 반복문 두 개가 중첩되어 있으므로, 시간복잡도는 모든 경우에 대해 $\Theta(n^2)$이다.

1.2. Insertion Sort

```java
    private static int[] DoInsertionSort(int[] value)
    {
        for (int i=1; i<value.length; i++) {
            for (int j=i; j>0; j--) {
                if (value[j] < value[j-1]) {
                    swap(value, j, j-1);
                } else {
                    break;
                }
            }
        }

        return (value);
    }
```

Insertion Sort 에서는 배열의 0 부터 i-1 번째까지의 값들이 정렬되어있고, i 번째 값을 앞의 정렬된 부분의 적절한 위치에 삽입한다. 위 코드에서 i 는 value.length-1 까지 증가하고, 비교되는 값의 인덱스인 j 는 i 부터 1 까지 감소한다. 배열의 j 번째 값과 j-1 번째 값이 비교된다. j 번째 값이 j-1 번째 값보다 작으면 둘을 교환하고, 아니면 j 에 대한 반복문이 종료된다. 반복문 두 개가 중첩되어 있으므로 시간복잡도는 평균적인 경우와 최악의 경우에 $\Theta(n^2)$이다. 그러나 최선의 경우에는 이미 배열이 정렬되어있어 j=i 일때 각 반복문이 종료되므로, 시간복잡도가 $\Theta(n)$이다.

1.3. Heap Sort

Heap 클래스 구현은 자료구조 강의 노트를 참고하였다. Heap Sort 는 주어진 배열을 buildHeap()을 통해 heap 구조로 만들고, max heap 을 이용한 경우 deleteMax()로 최댓값을 하나씩 배열의 뒤부터 쌓는다. Max heap 의 root 에는 항상 그 배열의 최댓값이 있고, deleteMax()로 그 값을 제거하더라도

percolateDown()이 호출되어 다시 heap 구조로 복원된다. buildHeap 에서는 (n-2)/2 번의 percolateDown()이 호출되며, 시간복잡도는 $\Theta(n)$ 이다. deleteMax()의 시간복잡도는 최악의 경우 $\theta(\log n)$, 최선의 경우 $\Theta(1)$이다. 이 과정이 n 번 일어나므로 평균적인 경우와 최악의 경우 시간복잡도는 $\Theta(n\log n)$이고, 최선의 경우 $\Theta(n)$이다.

```java
    private static int[] DoHeapSort(int[] value)
    {
        Heap heapArr = new Heap(value);
        heapArr.buildHeap();
        try {
            for (int i=value.length-1; i>=0; i--) {
                value[i] = heapArr.deleteMax();
            }
        } catch (Exception e) {
            System.err.println("index error occured in deleteMax()");
        }

        return (value);
    }
```

1.4. Merge Sort

```java
    private static void mergeSort(int[] value, int[] res, int p, int r) {
        int size = r - p + 1;

        // basecase
        if (size < 2) {
            return;
        }

        // division & recursion
        int mid = p + size / 2;
        mergeSort(res, value, p, mid-1);
        mergeSort(res, value, mid, r);

        // mergence
        int i = p;
        int j = mid;
        int k = p;
        while ((i<=mid-1) && (j<=r)) {
            if (value[i] < value[j]) {
                res[k++] = value[i++];
            } else {
                res[k++] = value[j++];
            }
        }
        while (i <= mid-1) {
            res[k++] = value[i++];
        }
        while (j <= r) {
            res[k++] = value[j++];
        }
    }
```

뼈대코드에 mergeSort()라는 함수를 따로 만들었다. value 는 정렬되는 배열, res 는 정렬 결과가 저장되는 배열, p 는 시작 인덱스, r 은 끝 인덱스다. 이 함수를 호출하기 전 미리 value 를 res 에 복제하였다. Basecase 는 size 가 2 보다 작은 경우다. mid = p + size / 2 로 계산하여 배열을 두 부분으로 나누고, res 와 value 의 순서를 바꾸어 재귀호출을 하였다. 이후에는 value 의 mid 를 기준으로 하는 두 부분이 각각 정렬된다. 임시 배열에서 결과 배열로 모든 값을 복사하는 과정, 또는 새로운 배열을 동적 선언할때마다 생기는 오버헤드를 줄이기 위해 이와 같이 구현하였다. 정렬된 두 부분의 각 값을 비교하여 작은 값부터 res 의 앞에 삽입한다. 둘 중 한 부분에서 삽입되지 않은 값도 마저 res 에 삽입되면, 최종적으로 res 는 정렬된 배열이 된다. 시간복잡도는 $\Theta(n\log n)$이다.

1.5. Quick Sort

Quick Sort 는 Merge Sort 와 비슷하게 분할-정복 방법을 이용하는 정렬 알고리즘이다. Basecase 는 size 가 2 보다 작은 경우다. 배열의 마지막 원소(value[r])를 pviot 로 설정하였다. 반복문의 각 단계에서, pivotIdx 는 배열에서

```java
    private static void quickSort(int[] value, int p, int r) {
        int size = r - p + 1;

        // basecase
        if (size < 2) {
            return;
        }

        // partition
        // pivot = value[r]
        int pivotIdx = p;
        for (int i=p; i<r; i++) {
            if (value[i] < value[r]) {
                swap(value, i, pivotIdx++);
            }
        }
        swap(value, pivotIdx, r);

        // recursion
        quickSort(value, p, pivotIdx-1);
        quickSort(value, pivotIdx+1, r);
    }
}
```

pivot 보다 작다는 것이 확인된 마지막 값의 바로 다음 인덱스를 가리키고, i 는 pivot 보다 크거나 같다는 것이 확인된 마지막 값의 인덱스를 가리킨다. pivotIdx 를 기준으로 배열은 두 부분으로 나뉘는데, 하나는 pivot 보다 작은 값만 있고 나머지는 pivot 보다 크거나 같은 값만 있다. 각각에 대해 재귀호출을 하면 배열이 정렬된다. 시간복잡도는 평균적인 경우와 최선의 경우 $\Theta(n\log n)$이고, 이미 정렬되어있는 최악의 경우에는 $\Theta(n^2)$이다.

1.6. Radix Sort

```java
    private static int[] DoRadixSort(int[] value)
    {
        final int INT_DIGITS = (int) Math.log10(Integer.MAX_VALUE) + 1;
        int size = value.length;
        int[] count = new int[19];                      // -9 ~ 9
        int[] tmp = new int[size];

        for (int i=0; i<INT_DIGITS; i++) {
            for (int j=0; j<19; j++) {                  // count 초기화
                count[j] = 0;
            }
            int num = (int) Math.pow(10.0, (double) i); // num = 1, 10, 100, ...
            boolean quit = true;
            for (int j=0; j<size; j++) {
                int digit = (value[j] / num) % 10;
                count[digit+9]++;
                if (digit != 0) {                       // 모든 digit이 0이면 반복문 종료
                    quit = false;
                }
            }
            if (quit) {
                break;
            }
            for (int j=1; j<19; j++) {                  // count의 누적합 저장
                count[j] += count[j-1];
            }
            for (int j=size-1; j>=0; j--) {             // 정렬된 결과를 tmp에 저장
                int digit = (value[j] / num) % 10;
                tmp[count[digit+9]-1] = value[j];
                count[digit+9]--;
            }
            value = tmp.clone();
        }

        return (value);
    }
```

Radix Sort 는 정수의 각 자릿수에 대해서 Counting Sort 로 정렬하는 것을 반복한다. 정렬되는 데이터로 음수가 허용되므로 count 의 크기는 인덱스 -9~9 를 이용하기 위해 19 로 설정하였다. 일의 자리 부터 더 높은 자리를 향해 Counting Sort 가 반복된다. 해당하는 자릿수의 숫자가 나오는 횟수가 count 에 저장된다. count 의 가중합을 구하면 해당 자릿수에 대해 정렬된 배열을 구할 수 있다. 이는 tmp 에 저장되므로, 정렬되는 결과가 들어갈 배열인 value 로 복사한다. Radix Sort 는 stable sort 이어야 하므로, tmp 의 인덱스 j 는 size-1 에서 1 씩 감소하도록 하였다. 시간복잡도는 정수의 최대 자릿수를 d 라고 하면 $\Theta(dn)$이다.

2. 정렬 소요 시간 분석

정렬 알고리즘	N = 50000, -30000~30000, 10 회, ms				N = 100000, -60000~60000, 5 회, ms			
	평균	표준편차	최소	최대	평균	표준편차	최소	최대
Bubble	3042.5	92.7	2867.0	3173.0	11539.8	154.2	11395.0	11783.0
Insertion	318.0	7.9	311.0	337.0	2659.8	786.0	1267.0	3120.0
Heap	7.0	0.6	6.0	8.0	14.6	0.5	14.0	15.0
Merge	5.0	0.5	4.0	6.0	12.2	0.8	11.0	13.0
Quick	4.6	0.8	4.0	6.0	9.8	0.4	9.0	10.0
Radix	3.7	0.5	3.0	4.0	8.0	0.7	7.0	9.0

입력이 “r 50000 -30000 30000”인 경우 10 회, “r 100000 -60000 60000”인 경우 5 회 정렬을 반복하였다. 각 알고리즘에 대해서 정렬 소요 시간의 세번째 이후 결과를 기록하였다. 모든 경우에 대해서 Bubble Sort 가 가장 느렸고, Radix Sort 가 가장 빨랐다. 두 경우 모두에 대해서 Quick Sort 가 Merge Sort 보다 빨랐다.